

សាកលវិទ្យាល័យ អាស៊ី អឺរ៉ុប

មហាវិទ្យាល័យ វិទ្យាសាស្ត្រ និងបច្ចេកវិទ្យា



មុខវិជ្ជា៖

Programming Methodology in C++

មេរៀនទី៧៖

ជំពូក ៧ Classes And Objects

ឆ្នាំទី ១ ឆមាស ទី ២

គណៈកម្មការអភិវឌ្ឍន៍កម្មវិធីសិក្សា

© 2024

មតិកា

- ❑ 1. ការបង្កើត និង ប្រើប្រាស់ Classes និង Objects
- ❑ 2. ការប្រើប្រាស់ Constructor និង Destructor
- ❑ 3. ការប្រើប្រាស់ Pointer to Class
- ❑ 4. ការប្រើប្រាស់ Class Defined with Structure , Union
- ❑ 5. ការប្រើប្រាស់ Overloading the Operators, Static Member
- ❑ 6. ការប្រើប្រាស់ FriendShip និង Inheritance
- ❑ 7. ការប្រើប្រាស់ Single និង Multiple Inheritance
- ❑ 8. ការប្រើប្រាស់ Pointer to Base Class Abstract Base Class

មតិកា

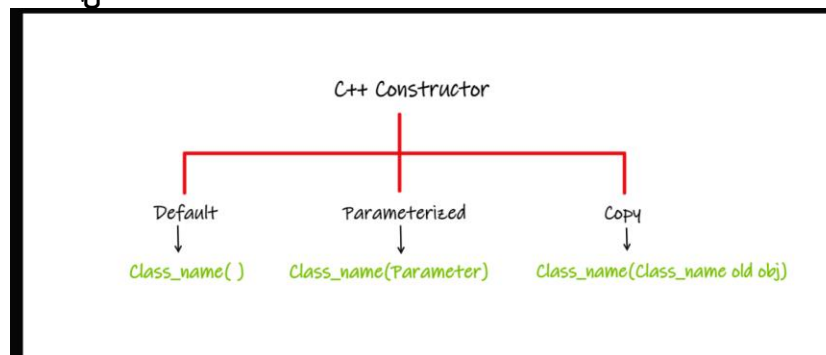
- ❑9. ការប្រើប្រាស់ Function Template និង Class Template
- ❑10. ការប្រើប្រាស់ Virtual Member , Template Specialization
- ❑11. ការប្រើប្រាស់ NameSpace និង ដោះស្រាយ Exception
- ❑ 12. ការប្រើ ការបំបែកប្រភេទទិន្នន័យ
- ❑១៣. ការកំណត់ Preprocessor Directives ការប្រើ Input/ output with Files.
- ❑14. ការប្រតិបត្តិរបៀប Read និង Write Files

1. ការប្រើប្រាស់ Constructor និង Destructor

- 1.១. អ្វីទៅ ហៅថា Constructor?
- ១.២. ការប្រើប្រាស់ Constructor .
- ១.3. អ្វីទៅ ហៅ ថា Destructor?
- 1.4. ការប្រើប្រាស់ Destructor.
- Example : Coding .

១.1. អ្វីទៅ ហៅថា Constructor?

- **Constructor** គឺ method ពិសេសមួយក្នុង C++ ដែលត្រូវបានប្រើដើម្បី Initialize Object នៃ Class។ Constructor ត្រូវបានកោះហៅដោយស្វ័យប្រវត្តិនៅពេលដែល Object ត្រូវបានបង្កើត ។
- **Constructor** យើងអាចបង្កើត បាន ដោយមានការសម្គាល់ គឺ មានឈ្មោះ ដូច Class និង មានរង្វង់ក្រចកផង ដែល()
- Constructor មានដូចជា ៖



Constructor

- **Default Constructor** គឺ method មិនមាន parameter ដែលត្រូវបានកោះហៅដោយស្វ័យប្រវត្តិនៅពេល Object ត្រូវបានបង្កើតដោយមិនមាន argument។
- **Parameterized Constructor** គឺ Constructor ដែលមាន Parameter ។ វាត្រូវបានប្រើដើម្បី Initialize អថេរតាមតម្លៃដែលបានបញ្ជាក់។
- **Copy Constructor** គឺ Constructor ដែលប្រើដើម្បីបង្កើត Object ថ្មីពី Object ដែលមានរួចហើយ។ វាត្រូវបានប្រើដើម្បីចម្លងតម្លៃពី Object មួយទៅ Object មួយទៀត។

លក្ខណៈពិសេសនៃ Constructor

- ឈ្មោះដូចនឹង Class : Constructor ត្រូវមានឈ្មោះដូចនឹង Class ដែលវាស្ថិតក្នុងនោះ។
- គ្មាន Return Type : Constructor មិនមាន Return Type (ទោះបីជាមិនមាន void ក៏ដោយ)។
- កោះហៅដោយស្វ័យប្រវត្តិ : Constructor ត្រូវបានកោះហៅដោយស្វ័យប្រវត្តិនៅពេលដែល Object ត្រូវបានបង្កើត។
- អាចមាន Parameter : Constructor អាចទទួល Parameter ដើម្បី Initialize អថេរ។ នេះគឺជាលក្ខណៈនៃ Parameterized Constructor។

លក្ខណៈពិសេសនៃ Constructor

- អាចទទួលបាន Default Constructor : បើគ្មាន Constructor ដែលបានបញ្ជាក់រួចមក ពេលនោះ C++ ទំនងនឹងបង្កើត Default Constructor ដោយស្វ័យប្រវត្តិ។
- អាចត្រូវបាន Overload : Constructor អាចត្រូវបាន Overload ដោយមាន Parameter ខុសគ្នា។
- Copy Constructor : លក្ខណៈពិសេសមួយដែលត្រូវបានប្រើដើម្បីចម្លង Object មួយទៅ Object ថ្មី។
- អាចប្រើជាមួយ Dynamic Memory Allocation : Constructor អាចប្រើប្រាស់ជាមួយ Dynamic Memory Allocation ដើម្បី Initialize អថេរនៅលើ Heap។

Syntax

- Default Constructor

```
class ClassName {  
public:  
    ClassName() {  
        // Initialization code  
    }  
};
```

Syntax

- Parameterized Constructor

```
class ClassName {  
public:  
    ClassName(DataType param1, DataType param2) {  
        // Initialization code using param1 and param2  
    }  
};
```

Syntax

- Copy Constructor

```
class ClassName {  
public:  
    ClassName(const ClassName &obj) {  
        // Initialization code using obj  
    }  
};
```

Syntax

- Constructor with Initializer List

```
class ClassName {  
    int x;  
    int y;  
public:  
    ClassName(int a, int b) : x(a), y(b) {  
        // Additional initialization code (if any)  
    }  
};
```

Syntax

- Default Arguments in Constructor

```
class ClassName {  
    int x;  
public:  
    ClassName(int a = 0) : x(a) {  
        // Initialization code  
    }  
};
```

Constructor Coding

```

1 // Online C++ compiler to run C++ program online
2 #include <iostream>
3 using namespace std;
4
5 class Example {
6     int x;
7     int y;
8 public:
9     // Default Constructor
10    Example() {
11        x = 0;
12        y = 0;
13    }
14
15    // Parameterized Constructor
16    Example(int a, int b) {
17        x = a;
18        y = b;
19    }
20
21    // Copy Constructor
22    Example(const Example &obj) {
23        x = obj.x;
24        y = obj.y;
25    }
26
27    void display() {
28        cout << "x: " << x << ", y: " << y << endl;
29    }
30 };
31
32 int main() {
33     Example obj1;           // Calls Default Constructor
34     Example obj2(10, 20);   // Calls Parameterized Constructor
35     Example obj3 = obj2;    // Calls Copy Constructor
36
37     obj1.display();
38     obj2.display();
39     obj3.display();
40
41     return 0;
42 }

```

```

/tmp/sZhn72BaaJ.o
x: 0, y: 0
x: 10, y: 20
x: 10, y: 20

```

=== Code Execution Successful ===



១.3. អ្វីទៅ ហៅ ថា Destructor ?

Destructor ៖ គឺជា function ពិសេសមួយនៅក្នុង C++ ដែលត្រូវបានកោះហៅនៅពេលដែល Object ត្រូវបានលុបចោល ឬចេញពីសិទ្ធិរបស់វា។

Destructor មានលក្ខណៈសំខាន់ៗដូចខាងក្រោម៖

- ឈ្មោះដូចនឹង Class បន្ថែមនឹង ~ ខាងមុខ
- មិនមាន Parameter ទេ
- មិនមាន Return Type ទេ (ទោះបីជាមិនមាន void ក៏ដោយ)
- ត្រូវបានកោះហៅដោយស្វ័យប្រវត្តិនៅពេល Object ត្រូវបានលុបចោល

1.4. ការប្រើប្រាស់ Destructor

Syntax របស់ Destructor :

```
class ClassName {  
public:  
    ~ClassName() {  
        // Cleanup code  
    }  
};
```


Coding Destructor

```

1 // Online C++ compiler to run C++ program online
2 #include <iostream>
3 using namespace std;
4
5 class Example {
6     int *ptr;
7 public:
8     // Constructor
9     Example(int a) {
10         ptr = new int;
11         *ptr = a;
12         cout << "Constructor called! Value: " << *ptr << endl;
13     }
14
15     // Destructor
16     ~Example() {
17         delete ptr;
18         cout << "Destructor called! Memory released." << endl;
19     }
20 };
21
22 int main() {
23     Example obj(10); // Constructor will be called here
24     // Destructor will be called automatically when obj goes out of scope
25     return 0;
26 }
27

```

```

/tmp/1h0C8a4ste.o
Constructor called! Value: 10
Destructor called! Memory released.

```

```

=== Code Execution Successful ===

```

ភាពខុសគ្នា រវាង Constructor និង Destructor

Constructor	Destructor
Constructor ហៅពេល object បានបង្កើត	Destructor ហៅពេល object បាន បាត់បង់ទៅ ឬ លុបចោល
Constructor ទទួលយក Parameters	មិន ទទួលយក Parameters
Constructor អាច Overloaded	មិនអាច Overloaded
អាច Constructors បានច្រើន	ពួកវាមានតែ Destructor



សាកលវិទ្យាល័យ អាស៊ី អឺរ៉ុប

ASIA EURO UNIVERSITY

សិក្សាស្រាវជ្រាវ គ្រប់ទីកន្លែង

អរគុណ

សម្រាប់ការយកចិត្តទុកដាក់

<https://elearning.aeu.cloud>